

System Verilog versus UVM-based Verification of AXI4-Lite Arbitration

Aiswariya Ramesh Babu
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
aiswariya.rameshbabu@sjsu.edu

Pooja Anand
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
pooja.anand@sjsu.edu

Youngsoo Kim
Dept. of Electrical and Computer Engineering
Bradley University
Peoria, USA
ykim@bradley.edu

Shrikant Jadhav
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
shrikant.jadhav@sjsu.edu

Abstract—

The communication bus is one of the critical components in System-of-Chip design. The on-chip bus communication architecture impacts the overall performance of an SoC design. As design complexity increases, they become more error-prone, and verification becomes challenging. According to a few studies, verification consumes about 70% of the total development time and is considered a bottleneck in the ASIC design cycle. This paper focuses on the design and verification of the AXI4-Lite arbitration algorithm. We present a design and implementation of a high-performance AXI4-Lite interconnect using System Verilog (SV) to connect three managers and six subordinate devices. We performed the verification of an AXI4-Lite arbitration system using two methodologies i.e., SV and Universal Verification Methodology (UVM). Several test cases were applied to check all possible scenarios for arbitration verification. The SV and UVM verification techniques are compared in this paper, and it is observed that the UVM methodology is a better choice for verifying large and complex designs.

Index Terms—AXI4-Lite, verification, arbitration, Universal Verification Methodology (UVM), System Verilog (SV).

I. INTRODUCTION

Modern SoC designs are predominantly becoming communication-centric, as the reusable components like processors, memories, etc., are connected with custom hardware components and interfaces through an on-chip bus communication fabric to meet various design constraints like cost, performance, area, power, and reliability [1]. ARM developed Advanced Microcontroller Bus Architecture (AMBA), an open-standard, on-chip interconnect specification for connecting and managing the different functional blocks in System-on-Chip (SoC) designs. AMBA bus architectures facilitated the development of multi-processors with multiple components and controllers. Since its inception in 1996, the scope of AMBA architectures has extended beyond microcontrollers. The primary design principle of AMBA is to reduce the silicon infrastructure while enabling high-performance and low-power on-chip communication. Owing to its well-defined specifications, AMBA is ubiquitously

employed in a broader spectrum of Application Specific Integrated Circuits (ASIC) and SoC components, including the application processors found in modern, sophisticated devices like smartphones, smart watches, laptops, etc.

A design challenge is involved when multiple devices are connected to the Advanced eXtensible Interface (AXI) 4 interface. All devices may request access to the bus simultaneously, leading to contention and latency in accessing the bus at the right time. This affects the system's overall performance. The concept of arbitration was introduced to overcome this design challenge and enhance design performance. Arbitration is the technique used to address the communication bottleneck and resolve the problem of bus contention when multiple devices simultaneously need access to the bus. The arbitration mechanism involves an arbiter that allows access to only one manager at a time. There are different kinds of arbitration algorithms, i.e., fixed priority arbitration, round-robin arbitration, priority round-robin arbitration, etc. The arbiter observes all the bus requests from different managers. Based on the arbitration algorithm implemented, the arbiter will grant access to one manager.

As the design complexity increases exponentially, verification becomes challenging. Verification is considered a critical bottleneck in the semiconductor industry, as it consumes about 70% of the total design development time and directly influences the time to market [2]. The ability to deliver products on time might give the business a competitive edge in the market and increase profits. Since verification is time-consuming, verification engineers must use advanced verification methodologies to speed up the verification process by re-using the test environments. If the designs are not rigorously tested, there may be bugs in the design that are discovered only after the tape-out. This causes the company to lose millions of dollars. Therefore it is referred to as a billion-dollar mistake. For example, in 1994, a hardware bug was found in Intel's Pentium that affected the floating-point division unit. In December 1994, Intel recalled all the defective

chip processors and subsequently incurred a loss of about \$475 million [3]. This proves the importance of pre-silicon and post-silicon verification.

This paper presents an efficient arbitration mechanism design for the AXI4-Lite interconnect using System Verilog. We also developed the UVM environment to verify the design. AXI4-Lite is a subset of AXI and is one of the most widely used on-chip communication protocols in SoC designs. System Verilog is used to implement the design of the AXI4-Lite interconnect, which connects three managers and six subordinates to demonstrate arbitration. The design's performance and efficiency are optimized using the priority round-robin arbitration algorithm. UVM test bench hierarchy is developed to verify the arbitration of AXI4-Lite. The design of AXI4-Lite arbitration will serve as a foundation for designing arbitration algorithms for different AMBA on-chip bus protocols. Since UVM is used to verify the design, most test bench components like environment, monitor, sequences, and test bench stimuli can be reused across different projects. Furthermore, this will help improve the overall verification effort, reducing the market time.

II. RELATED WORK

In modern SoC designs, effective communication is challenging and is considered a bottleneck. According to Makni et al. [4], on-chip bus communication architectures are crucial in determining the performance of SoC designs. As design complexity increases, the conventional SoC interconnects cannot meet the tremendous requirements of modern SoC designs. Hence, efficient on-chip bus communication architectures were developed to enable high-performance data transfer. The authors highlighted the three different protocols of AXI – stream, burst, and lite and the parameters to be considered while selecting the correct interface for the application.

Arbitration is crucial in a multi-manager and multi-subordinate system to resolve bus contention and determine the manager that can access the bus when multiple managers request bus access simultaneously. There are different arbitration techniques. Noami et al. [5] designed an AXI interconnect with four managers and one subordinate using a static fixed-priority scheduling algorithm. The priority is based on the burst length. The manager with the least burst length prioritizes access to the bus. The SoC verification of the design is implemented on an FPGA ZedBoard using the Tcl console commands on the Vivado tool.

According to Shrivastava et al. [6], on-chip bus communication architectures should be flexible as they play a critical role in determining the SoC's overall performance. The authors proposed a high-bus utilization arbitration technique that allowed each manager fixed bandwidth per time frame. They reviewed the design challenges of arbitration techniques and compared the performance of these arbiters based on parameters such as average latency, average acceptance rate and waiting time of managers, and average shared bus bandwidth utilization by each manager.

With the reduction in process technology nodes and increased design constraints, SoC's are developed to model all the functional components on a single chip. The SoC interconnect connects an Intellectual Property (IP) core to the neighboring interfaces. Math et al. [7] designed an AMBA AXI4 protocol in Verilog Hardware Description Language (HDL) to perform data transactions on an SoC bus. The authors performed multiple read and write transactions using two test cases.

Verification is crucial in the design cycle as it determines the quality of the design. Gaikwad et al. [8] emphasized the verification challenges and the inefficiency of traditional methods in verifying large and complicated SoC designs. Deepu et al. [9] discussed the importance of verification as it consumed about 70% of the development time. The authors designed a System Verilog test bench to validate the data transactions of the AXI3 protocol. Similar to [9], Mahesh et al. [10] designed a System Verilog test bench environment to verify the memory transactions on the AXI protocol using the Mentor Graphics Questa Simtool. The significant difference is that later used random constraint generation to generate complex transactions.

UVM is the current verification methodology standard based on System Verilog constructs, such as object-oriented concepts, that allows the generation of more flexible tests with reusable components [11, 12]. As design complexity increases exponentially, traditional verification methods using Verilog become obsolete as they lack the test bench environment's flexibility, reusability, and scalability. This reduces the time and effort to generate complex test-bench environments and facilitates quicker time to market. The authors designed a UVM-based Verification IP for AXI to verify the read and write transactions from the same address location with the same parameters.

The test-bench scalability is constrained when the verification environment is modeled using System Verilog, which cannot support complicated and more extensive designs. This paper presents the implementation of the design and verification of AMBA AXI4-Lite arbitration using System Verilog and UVM test bench environment. Even though [11] and [12] have implemented the verification environment in UVM, their test bench environment is designed to verify the AMBA AXI4/AXI4-Lite protocol when connected to a single manager and single subordinate device. However, in contrast, this design includes the connection of three managers and six subordinates, making it more complex. [5] implemented a static fixed-priority scheduling algorithm in the design, whereas this design employs priority round-robin arbitration to improve performance. Due to UVM's scalability, modularity, and reusability, complex designs can easily be verified. Hence, our approach facilitates the comparison of System Verilog and UVM verification techniques and demonstrates that the UVM test bench environment is a better choice for verifying larger and more complex designs.

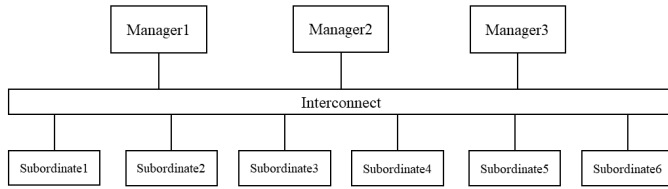


Fig. 1: AXI4-Lite arbitration design with multiple managers and subordinates

III. HARDWARE IMPLEMENTATION OF AXI4-LITE ARBITRATION

Figure 1 presents the AXI4-Lite arbitration design with three managers and six subordinates connected to the interconnect. The managers and subordinates are interfaced to a single shared bus. All the devices and functional blocks are served using the shared bus. However, the disadvantage of the shared bus is that only a single device can be served at a time. The design of AXI4-Lite arbitration consists of a protocol signal package, signal interface, three manager modules, six subordinate modules, interconnect logic, arbitration definition, decoder module, and memory units. The behavioral description of the design specification is implemented inside the module, which is a fundamental descriptive unit of Verilog.

The transaction between the manager and subordinate is a collection of valid addresses and data channel information. The exchange of data occurs when the *VALID* and *READY* signals are asserted. The *interface* is used to implement AXI4-Lite arbitration to define all the five independent channels for the protocol, i.e., read address, read data, write address, write data, and write response. The System Verilog interface construct encapsulates the signals required for the design under a single name, thus enabling reuse and maintainability. The *modport* construct is used in the interface to provide the signal directions for the module ports.

We implemented the Finite State Machines (FSM) to define the behavior of AXI4-Lite arbitration, as shown in Figure 2. The FSM consists of six states - the reset condition known as the idle state and one state each for the five independent channels inside the bus. As shown in the figure, whenever the reset is set, the bus signals return to the idle state, where the FSM is ready to accept the read or write controls and information. When any of the managers start to send the controls on read transactions, it indicates a valid address for read to happen to the subordinate mentioned in the read address channel. This moves the state machine to the *s_raddr* state. The state machine moves to the next state *s_rdata* when *ARVALID* and *ARREADY* signals are asserted. The subordinate reads the data and updates it in the read data channel. After the read data transaction is completed, the *RVALID* and *RREADY* signals are asserted to indicate the transaction for the read request between the corresponding manager and subordinate is complete, and the state machine goes back to the *idle* state.

When any of the managers start to send the controls on write

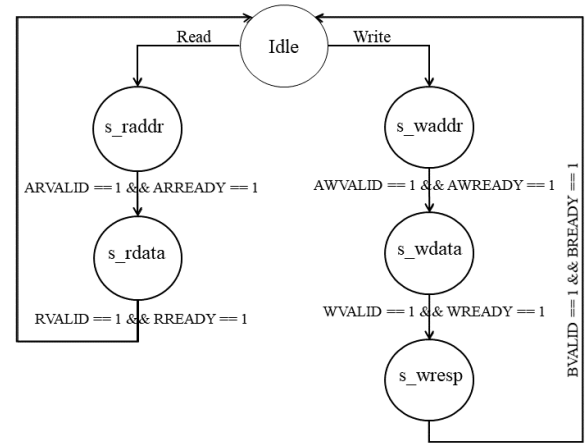


Fig. 2: FSM for AXI4-Lite Design

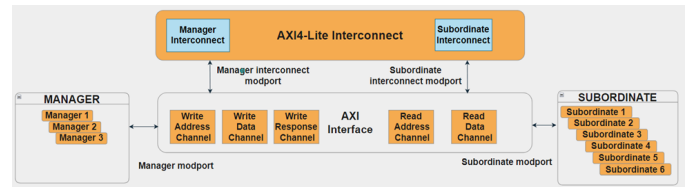


Fig. 3: Block Diagram of AXI4-Lite Arbitration Design

transactions, it indicates a valid address for write to happen to the subordinate mentioned in the write address channel. This moves the state machine to the *s_waddr* state. The state machine moves to the next state *s_wdata* when *AWVALID* and *AWREADY* signals are asserted. Here the subordinate writes the valid data from the write data channel. Once the data is written, the state machine waits for the assertion of *WVALID* and *AREADY* signals and then moves to the *s_wresp* state. After the write transaction is completed, the *BVALID* and *BREADY* signals are asserted to indicate that the transaction for the write request between the corresponding manager and subordinate is complete, and the state machine moves back to the idle state.

Figure 3 presents the block diagram of AXI4-Lite arbitration. As shown in the figure, it includes an AXI4-Lite interface with modports for the five independent channels, an interconnect with an arbiter and a decoder, three manager modules, and six subordinate modules. The design also includes a package module with all the definitions and parameters.

We implemented the priority round-robin arbitration, one of the popular arbitration algorithms. The AXI4-Lite interconnect includes an arbiter. Arbitration is the process of implementing a scheduling algorithm. The arbiter is crucial to decide the manager that can access the bus and act as a serving point. In our AXI4-lite design, round-robin arbitration is implemented for time scale-based scheduling. Here, when three managers are used, manager one has the highest preference, followed by manager two and then manager three. The basic design is the fixed time slot-based logic. This increases bus performance but at the expense of the managers' ability to use the bus

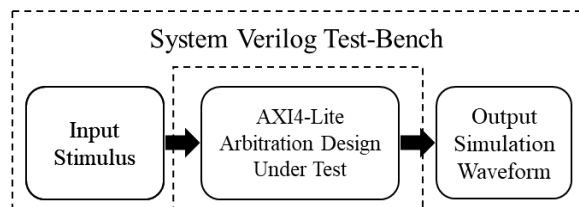


Fig. 4: System Verilog Verification Test-Bench

reasonably.

In the FSM, if the manager waits for more than two cycles for both read and write transactions, it transits back to the *idle* state. In other words, when multiple managers request access to the bus simultaneously, the arbiter picks the manager with the highest priority. This process takes two clock cycles. Hence, the other managers who lost the arbitration return to the *idle* state, while the manager who won the arbitration successfully transfers data.

IV. SV & UVM-BASED VERIFICATION OF AXI4-LITE ARBITRATION

A. SV-based Verification

Figure 4 depicts a System Verilog test bench used for AXI4-Lite verification purposes. The interface connects the test bench to the AXI4-Lite Arbitration Design Under Test (DUT). The test bench sends input stimulus patterns to the DUT and checks if the DUT generates the expected results, thus verifying the functionality of the design.

Since the design of AXI4-Lite arbitration involves three managers and six subordinates, we implemented a priority round-robin arbitration algorithm. The test bench is connected to the design through the interconnect's interface modports, and the three managers and six subordinates are instantiated in the test bench module. Hence, the input stimulus is generated in the round-robin method to verify the arbitration. The arbitration of AXI4-Lite is verified using five unique test cases.

The five different SV test cases for verifying AXI4-Lite arbitration are shown in Table I. The SV programming concept, "task", is used to define five different test scenarios – *test_write()*, *test_read()*, *test_read2()*, *test_read_new()*, and *test_write_new()*. Manager 0 has the highest priority, and manager 2 has the least priority. Manager 0 is writing, and manager 1 is reading in the first test case. Since AXI4-Lite has separate read and write channels, these tasks happen simultaneously without conflict. The second and third test scenario includes managers 1 and 2 reading and writing simultaneously. Since manager 1 has the highest priority compared to manager 2, manager 1 wins the arbitration to access the bus. All three managers read and write simultaneously in the fourth and fifth test cases. Manager 0 has the highest priority. Thus it gets access to the channel while manager 1 and 2 returns to an idle state.

In contrast, manager 2 returns to the *idle* state after waiting for more than 2 counts. In the third test scenario, managers 1 and 2 read simultaneously. Since manager 1 has a higher

priority, it wins the arbitration. The fourth test case scenario verifies the arbitration of write transactions when all three managers write simultaneously. Only manager 0 gets access to the bus, while the other two managers return to *idle* state after waiting for more than 2 counts. Similarly, the last test case verifies the arbitration when all three managers are reading concurrently.

B. UVM-based Verification

Table II shows the five UVM test cases used to verify the AXI4-Lite arbitration. In the first test case, manager 0 performs a single read transaction from subordinate 3. There is no bus contention. Manager 0 gets access to the read channel. In the second test case, different managers perform simultaneous reads and writes. AXI4-Lite has dedicated channels for read and write transactions. Manager 0 gets access to the read channel, and Manager 1 gets access to the write channel. In a third test case, two reads (manager 0 and 1) and one write (manager 2) are performed simultaneously. Manager 0 has a higher priority over manager 1 and wins the arbitration. Manager 0 gets access to the read channel. Manager 2 gets access to the write channel as there is no contention. All three managers perform simultaneous reads and writes in the fourth and fifth test cases. As manager 0 has the highest priority, it gets access to the read and write channel. Manager 1 and 2 goes to the idle state.

An essential mechanism of UVM that sets it apart from SV verification is UVM reporting. SV uses system functions and tasks such as *\$display*, *\$write*, *\$strobe*, and *\$monitor* for debugging. In contrast, UVM reporting offers a rich collection of messages that can mask or alter the message severity based on the desired environment condition. The UVM reporting are of four different types, which can be implemented with different verbosity levels. This robust mechanism facilitates quicker debugging, saving time and effort. The four UVM reporting macros are - *UVM_INFO*, *UVM_ERROR*, *UVM_WARNING*, and *UVM_FATAL*. All these macros display the messages on the simulator. The *UVM_FATAL* has the highest severity and exits the simulation, while *UVM_ERROR* has a medium severity and increments the global error count to keep track of the errors. In terms of verbosity, there are six verbosity levels, with each represented by a value. The default verbosity level is *UVM_MEDIUM*.

The UVM Verification environment for AXI4-Lite arbitration is represented in Figure 5. It includes all the basic UVM base classes such as *top*, *test*, *environment*, *agent*, *scoreboard*, *monitor*, *driver*, *sequencer*, *sequence*, and *sequence item*. The UVM test bench is connected to the DUT's interconnect through the virtual interface. The significance of each classes are explained in the following paragraphs.

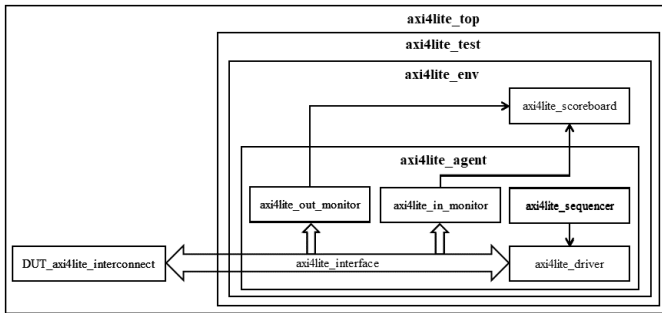
The *top* module instantiates DUT, verification components, and interfaces. It serves as the root of the UVM hierarchy. The manager, subordinate, interconnect, and interface modules are instantiated in the UVM testbench top. The *top* module includes the logic to create clocks, resets, and waveform dumps. The *test* is a component class extended from *uvm_test*.

TABLE I: SV Test Cases for AXI4-Lite Arbitration Verification

Test Case #	Description	Task	Arbitration
1	Random read and write from managers to different simultaneously	Manager 0 writes to subordinate 3 Manager 1 reads from subordinate 0	No bus contention. Since AXI4-Lite has dedicated channels for read and write transactions, they can happen simultaneously. Manager 0 gets access to the write channel bus and manager 1 gets access to the read channel bus.
2	Multiple writes simultaneously	Manager 1 writes to subordinate 0 Manager 2 writes to subordinate 4	Since manager 1 has the highest priority compared to manager 2, manager 1 wins the arbitration and gets access to the write channel bus.
3	Multiple reads simultaneously	Manager 1 reads from subordinate 0 Manager 2 reads from subordinate 4	Since manager 1 has a higher priority compared to manager 2, manager 1 wins the arbitration and gets access to the read channel bus.
4	All three managers write simultaneously	Manager 0 writes to subordinate 3 Manager 1 writes to subordinate 0 Manager 2 writes to subordinate 4	Since manager 0 has the highest priority, it wins arbitration and gets access to the write channel while the other two managers return back to the idle state after waiting for more than two cycles.
5	All three managers read simultaneously	Manager 0 reads from subordinate 3 Manager 1 reads from subordinate 0 Manager 2 reads from subordinate 4	Since manager 0 has the highest priority, it wins arbitration and gets access to the read channel while the other two managers return back to the idle state after waiting for more than two cycles.

TABLE II: UVM Test Cases for AXI4-Lite Arbitration Verification

Test Case #	Description	Task	Arbitration
1	Single read transaction	Manager 0 reads from subordinate 3	No bus contention. Manager 0 gets access to the read channel bus.
2	Simultaneous read and write from different managers	Manager 0 reads from subordinate 3 Manager 1 writes to subordinate 0	No bus contention. Since AXI4-Lite has dedicated channels for read and write transactions, they can happen simultaneously. Manager 0 gets access to the read channel and manager 1 gets access to the write channel.
3	Simultaneous two reads and one write	Manager 0 reads from subordinate 3 Manager 1 reads from subordinate 0 Manager 2 writes to subordinate 4	Since AXI4-Lite has dedicated channels for read and write transactions, one read and one write can happen simultaneously. Manager 0 has a higher priority than manager 1 and wins the arbitration. Manager 0 gets access to the read channel. There is no contention for the write channel, and manager 2 gets access to the write channel bus.
4	All three managers read simultaneously	Manager 0 reads from subordinate 3 Manager 1 reads from subordinate 0 Manager 2 reads from subordinate 4	Since manager 0 has the highest priority, it wins arbitration and gets access to the read channel. The other two managers return back to the idle state after waiting for more than two cycles.
5	All three managers write simultaneously	Manager 0 writes to subordinate 3 Manager 1 writes to subordinate 0 Manager 2 writes to subordinate 4	Since manager 0 has the highest priority, it wins arbitration and gets access to the write channel. The other two managers return back to the idle state after waiting for more than two cycles.

**Fig. 5: UVM Verification Environment for AXI4-Lite Arbitration**

The *test* class includes the logic to start the sequence using the “start” method. The *environment* is a container class that contains several reusable verification components like *agents* and *scoreboards*. It includes other smaller environments and optional top-level monitors depending upon the requirement. The *scoreboard* is a component class extended from the *uvm_scoreboard* base class and primarily contains checkers used to verify the functionality of the design. In a predicting test bench, the *scoreboard* also includes a reference/golden model to compare the actual results from the DUT with the predicted/expected results. The *agent* class is a container class

that encapsulates the UVM monitor, driver, and sequencer. It is classified into two types depending on its functionality – active agent and passive agent. The *monitor* is a component class extended from the *uvm_monitor* base class. The primary function of the *monitor* class is to observe the pin-level activity of the DUT and convert them into transactional-level activity for the *scoreboards*. The *driver* is a parametrized component class extended from the *uvm_driver* base class. The primary function of the *driver* class is to translate the data items from the sequence into pin-level activity for the DUT to process. The *sequencer* is a parametrized class extended from the *uvm_sequencer* base class. The *sequencer* acts as a FIFO that connects the *sequence* and *driver* classes. The *sequencer* includes an arbitration logic to pick one sequence and pass it to the driver when multiple sequences are connected simultaneously. The *sequence* is a parametrized object class that extends from the *uvm_sequence* base class. The *sequence* creates a set of test cases using the sequence items, which are subsequently sent to the *driver* through a *sequencer*. The *sequence* class includes several methods, macros, and pre-defined callbacks that are automatically called upon the start of the sequence in the *uvm_test*. The *sequence item* is an object class that extends from the *uvm_sequence_item* base class. The *sequence item* class contains the data fields necessary for creating the input stimulus. The data properties declared in the *sequence item* must be of type *rand* or *randc*, so they can be

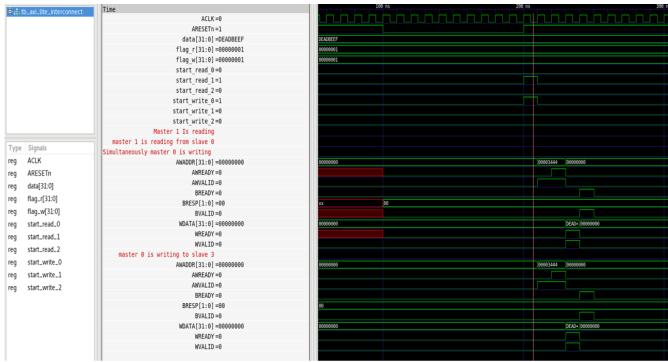


Fig. 6: SV Test Case 1 - Simultaneous read and write

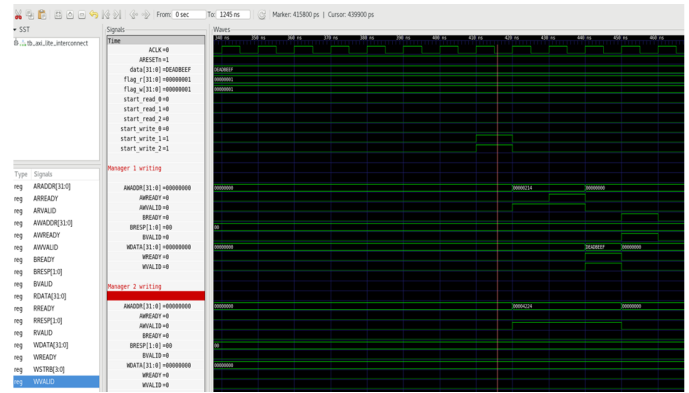


Fig. 7: SV Test Case 2 - Multiple writes from two managers

randomized while generating the stimulus.

V. EXPERIMENTAL RESULTS

We conducted experiments to evaluate the on-chip bus protocol arbitration. Synopsys's Verilog Compiler Simulator (VCS) Version R-2020.12 tool is used in designing and verifying AXI4-Lite arbitration. Three managers and six subordinate devices are used in the experiments. A priority round-robin arbitration algorithm is implemented. The waveforms are visualized and analyzed using the GTKWAVE tool. The design is verified using both SV and UVM. The verification plan involves the specifications' details and the detailed portrayal of all the situations to be considered in a unique but reusable coding style. Different test cases are designed to verify the complete RTL design to achieve better performance and more code coverage.

A. SV Test Bench Results

The simulation results for SV test case 1 are shown in Figure 6. Manager 0 writes to subordinate 3, and manager 1 reads from subordinate 0. The read and write flags are asserted. For manager 1, *start_read_1*, and for manager 0, *start_write_0* is set high. The address to read from and write to is sent out, and data corresponding to the address is loaded on the data bus. The AXI4-Lite has dedicated channels for read and write transactions. Manager 0 gets access to the write channel bus, and manager 1 gets access to the read channel bus. There is no bus contention, and simultaneous read and write take place, as shown in the figure.

Figure 7 presents the simulation results for SV test case 2. In test case 2, multiple writes are performed simultaneously. Manager 1 writes to subordinate 0, and manager 2 writes to subordinate 4 concurrently. For the write operations, the *start_write_1* and *start_write_2* signals are asserted. Since manager 1 has the highest priority compared to manager 2, manager 1 secures access to the write channel. Thus, as shown in the figure, manager 1 writes data to subordinate 0 (*WDATA*), and manager 2 remains idle.

The simulation results for SV test case 3 are shown in Figure 8. Simultaneous reads are performed in this test case. Manager 1 reads from subordinate 0 and manager 2 from subordinate 4 concurrently. *start_read_1* and *start_read_2*

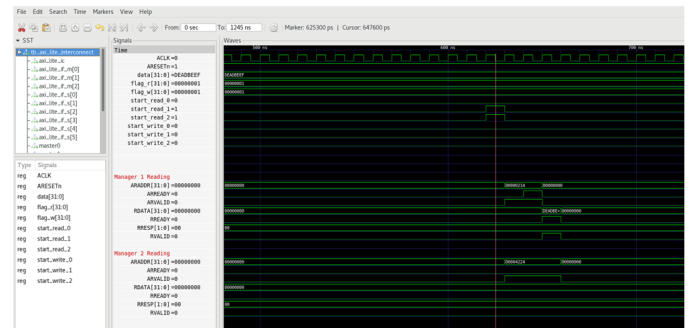


Fig. 8: SV Test Case 3 - Multiple reads from two managers

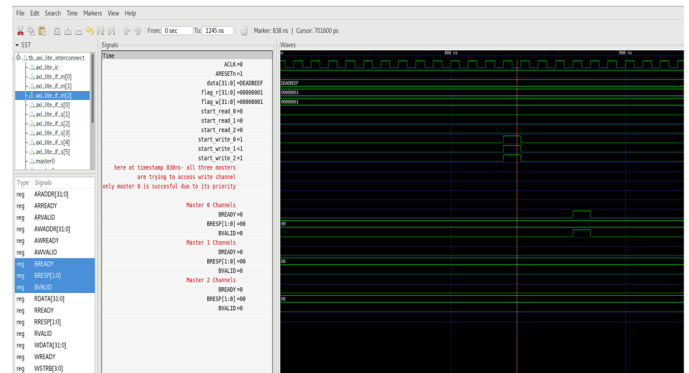


Fig. 9: SV Test Case 4 - Write channel arbitration

signals of managers 1 and 2 are set high. As per the priority algorithm, manager 1 gets access to the read channel over manager 2. As shown in the figure, manager 1 reads the data subordinate 0 (*RDATA*), and manager 2 remains idle.

The simulation results of write and read arbitration are shown in Figures 10 and 10, respectively. In SV test case 4, the write arbitration involves all three managers writing simultaneously to different subordinates. *start_write_0*, *start_write_1*, and *start_write_2* signals are asserted for a simultaneous write operation. In SV test case 5, the read arbitration involves all three managers reading simultaneously from different subordinates. *start_read_0*, *start_read_1*, and *start_read_2* signals are set high for a simultaneous read operation. Since manager

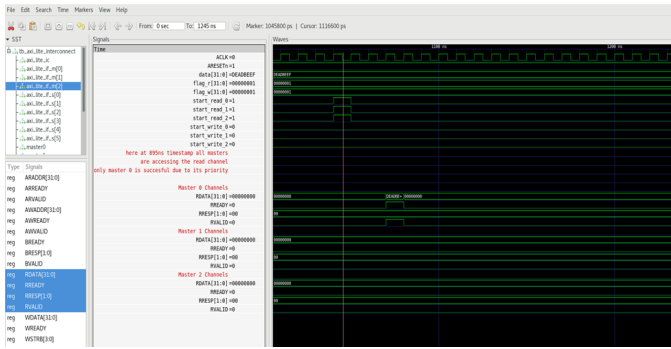


Fig. 10: SV Test Case 5 - Read channel arbitration

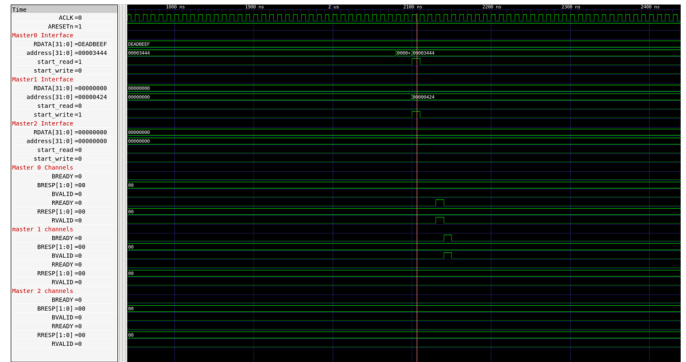


Fig. 12: UVM Test Case 2 - Simultaneous read and write

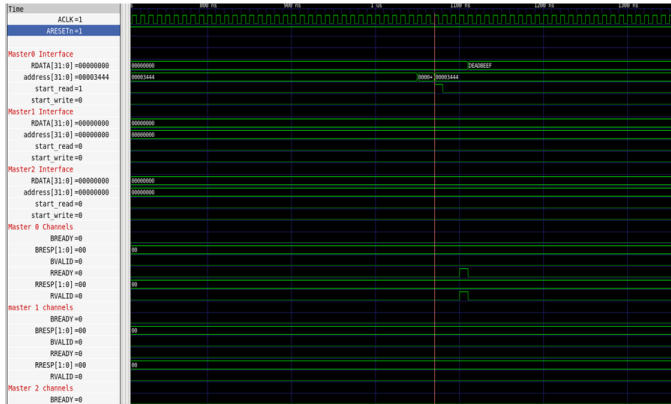


Fig. 11: UVM Test Case 1 - Only manager 0 is reading

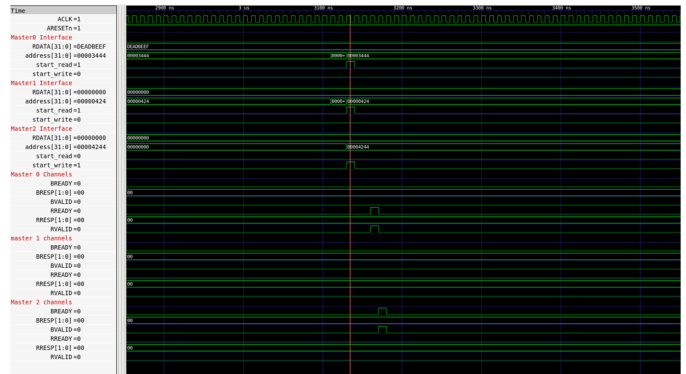


Fig. 13: UVM Test Case 3 - Simultaneous two reads and one write

0 has the highest priority, the priority round-robin arbitration algorithm enables manager 0 to access the read and write bus for test cases 4 and 5, as shown in the figure. Managers 1 and 2 return to an idle state.

B. UVM Test Bench Results

The simulation results for the UVM test case 1 are shown in Figure 11. Manager 0 reads from subordinate 3. The read flag *start_read* is set high. The address to read from is sent out, and the corresponding data is loaded on the data bus *RDATA*. There is no bus contention, and manager 0 gets access to the read channel, as shown in the figure.

Figure 12 presents the simulation results for UVM test case 2. In this test case concurrent read and write operations are performed. Manager 0 read from subordinate 3, and manager 1 writes to subordinate 0. The read and write flags are asserted. For manager 1, *start_write*, and for manager 0, *start_read* is set high. The address to read from and write to is sent out, and the corresponding data is loaded on the data bus. The AXI4-Lite has dedicated channels for read and write transactions. Manager 0 gets access to the read channel, and manager 1 gets access to the write channel. There is no bus contention, and simultaneous read and write take place, as shown in the figure.

The simulation results for UVM test case 3 are shown in Figure 13. Two reads and one write operation are performed

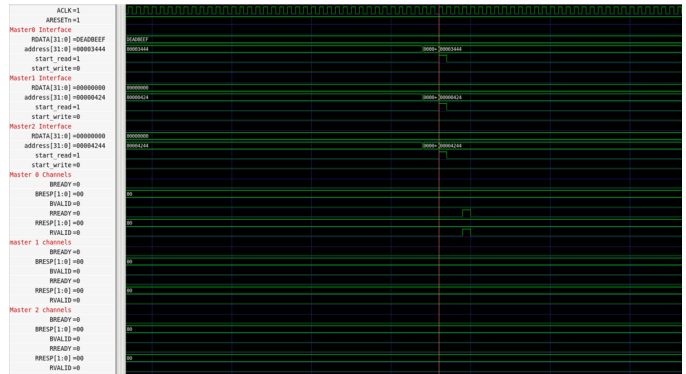


Fig. 14: UVM Test Case 4 - Read channel arbitration

simultaneously in this test case. Manager 0 reads from subordinate 3, manager 1 reads from subordinate 0, and manager 2 write to subordinate 4 concurrently. *start_read* signal for managers 0 and 1, and *start_write* signal for manager 2 is asserted. As per the priority algorithm, manager 0 gets access to the read channel over manager 1 for a read operation. Manager 2 gets access to the write channel since only one manager competed for the write bus. As shown in the figure, manager 0 reads the data subordinate 3 (*RDATA*), and manager 2 writes to subordinate 4.

Figures 14 and 15 present the simulation results for read and

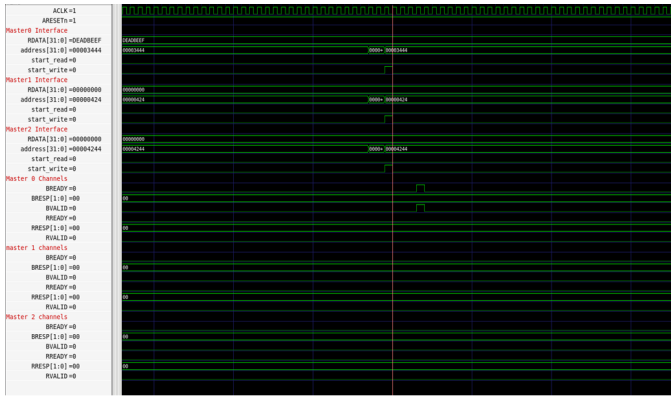


Fig. 15: UVM Test Case 5 - Write channel arbitration

write arbitration, respectively. In UVM test case 4, the read arbitration involves all three managers reading simultaneously from different subordinates. *start_read* signal is set high for a simultaneous read operation. Manager 0 reads from subordinate 3, manager 1 read from subordinate 0, and manager 2 reads from subordinate 4. In UVM test case 5, the write arbitration involves all three managers writing simultaneously to different subordinates. *start_write* signal is asserted for a simultaneous write operation. Since manager 0 has the highest priority, the priority round-robin arbitration algorithm enables manager 0 to access the read and write bus for test cases 4 and 5, as shown in the figure. Managers 1 and 2 return to an idle state.

VI. SV AND UVM TEST BENCH COMPARISON

The verification of AXI4-Lite arbitration using SV includes the tests and test bench in the same test bench module. The SV test bench lacks flexibility, modularity, reusability, and scalability to verify large and complex designs. The SV test bench environment has only one test bench module that instantiates the design and the virtual interface along with the test cases to verify the arbitration. The entire test bench has to be changed to verify another design with the same specification. In contrast, UVM has different components, such as sequence item, sequence, sequencer, driver, monitor, agent, scoreboard, environment, test, etc., to build a robust, reusable verification environment that separates the test cases from the test bench. In the UVM test bench environment, the test cases for verifying the arbitration are written in the sequence module, the tests are written in the test module, and the design and the verification modules are instantiated in the top module.

In UVM, only the driver and monitor must be changed to verify a different design with the same specifications. This is because the sequence is written with respect to the specifications and not the design. Hence, the sequence does not include any timings related to the DUT (Design Under Test). In contrast, the driver and monitor components are written in accordance with the design and contain the timings related to the design. This ensures that the sequence can be reused for different tests that have the same specifications, and the

driver and monitor can be changed. Furthermore, each task in UVM verification has a separate UVM class/module, such as a monitor for monitoring the signals, a sequence for generating test cases, a driver for driving the DUT signals, a scoreboard for comparing results, etc., that helps to ease debugging. This aids in saving verification time and effort, thus contributing to less time-to-market.

VII. CONCLUSION

The paper presents a design and verification of AXI-4 Lite arbitration. The Synopsys VCS tool was used to implement and verify the arbitration mechanism using SV and UVM. The design of AXI4-Lite arbitration is modeled in SV and includes three managers and six subordinate devices connected to the AXI4-Lite interconnect. The design successfully employs a priority round-robin arbitration algorithm and has two dedicated state machines for the read and write transactions, respectively. We verified the AXI4-Lite arbitration design using both SV and UVM test benches. The verification model of both SV and UVM included five different test cases each. The GTKWAVE tool was used to observe and analyze the simulation results and waveforms to verify the design's arbitration mechanism. Finally, the comparison of SV and UVM test benches demonstrated that UVM's modularity, flexibility, scalability, and reusability make it a better choice for verifying complicated designs. UVM facilitates the development of a robust, reusable verification environment that saves verification effort and time, thus positively impacting the time-to-market.

REFERENCES

- [1] S. Pasricha and N. Dutt, *On-Chip Communication Architectures: System on Chip Interconnect*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2008.
- [2] A. Molina and O. Cadenas, "Functional verification: Approaches and challenges," *Latin American Applied Research*, vol. 37, pp. 65–69, 2007.
- [3] E. Smith, "How a minor calculation error cost intel half a billion dollars," <https://www.vice.com/en/article/dyzqdj/how-a-minor-calculation-error-cost-intel-half-a-billion-dollars>.
- [4] M. Makni, M. Baklouti, S. Niar, and M. Abid, "Performance exploration of amba axi4 bus protocols for wireless sensor networks," in *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*, 2017, pp. 1163–1169.
- [5] A. Noami, B. P. Kumar, C. S. Paidimarry, N. Safi, and A. A. Alamamari, "Soc verification for the axi interconnect based on the static fixed-priority scheduling algorithm," in *2021 Fourth International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, 2021, pp. 1–6.
- [6] A. Shrivastava and S. K. Sharma, "Various arbitration algorithm for on-chip(amba) shared bus multi-processor soc," in *2016 IEEE Students' Conference on Electrical, Electronics and Computer Science (SCECS)*, 2016, pp. 1–7.
- [7] S. S. Math, R. B. Manjula, S. S. Manvi, and P. Kaunds, "Data transactions on system-on-chip bus using axi4 protocol," in *2011 INTERNATIONAL CONFERENCE ON RECENT ADVANCEMENTS IN ELECTRICAL, ELECTRONICS AND CONTROL ENGINEERING*, 2011, pp. 423–427.
- [8] N. Gaikwad and V. Patil, "Verification of amba axi on-chip communication protocol," in *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, 2018, pp. 1–5.
- [9] M. P. Deepu and R. Dhanabal, "Validation of transactions in axi protocol using system verilog," in *2017 International conference on Microelectronic Devices, Circuits and Systems (ICMDCS)*, 2017, pp. 1–4.
- [10] G. Mahesh and S. M. Sakthivel, "Verification of memory transactions in axi protocol using system verilog approach," in *2015 International Conference on Communications and Signal Processing (ICCSPP)*, 2015, pp. 0860–0864.
- [11] V. Melikyan, S. Harutyunyan, A. Kirakosyan, and T. Kaplanyan, "Uvm verification ip for axi," in *2021 IEEE East-West Design & Test Symposium (EWDTS)*, 2021, pp. 1–4.
- [12] H. Sangani and U. Mehta, "Uvm based verification of read and write transactions in axi4-lite protocol," in *2022 IEEE Region 10 Symposium (TENSYP)*, 2022, pp. 1–5.